

A Gated Architecture for Safe Cross-Session Memory in AI Coding Agents

Sam Gurung

ArtixNature Lab, San Francisco, CA

July 2026

admin@shikshya.ai

Abstract

AI coding agents increasingly pair conversational interfaces with code-generation pipelines: two subsystems with conflicting context requirements. Conversational memory is optimized for breadth, personalization, and recall; generation context is optimized for precision, relevance, and correctness. When conversational memory enters the generation pipeline unfiltered, irrelevant, stale, or contradictory signals can corrupt the generated output, a failure mode we term context poisoning. We propose the gated memory bridge (the “Memory PR” model), which adapts the pull-request paradigm to memory architecture: information crosses from conversation to generation only as typed, content-constrained, provenance-verified proposals that pass confidence-tiered review gates. We implement the extraction and validation layers of this architecture in a production system (Shikshya v2) and evaluate them across 18 extraction runs on six synthetic test sessions covering five scenarios. On this corpus, the extractor achieved 100% precision, 100% recall on required decisions, 100% provenance validation, and zero leakage of off-topic conversational content. The central empirical finding concerns tier self-assessment: the extracting model assigned the prohibited auto-approve tier in 20 of 25 restricted-type instances (80%), including every instance of the highest-risk type, and deterministic post-hoc enforcement improved overall tier accuracy from 35.5% to 93.5%, evidence that the proposing model should not be the sole authority over how its proposals are treated. Two pre-registered attempts to reproduce context poisoning at small scale returned null results, suggesting that poisoning risk may emerge with accumulated history rather than within short sessions; the gate meanwhile provides an 8.6× reduction in generation-context size that grows with conversation history. We additionally analyze five commercial systems (Cursor, Replit Agent, Claude Code, Bolt, Lovable) and identify the architectural decisions that expose or protect against context poisoning.

Keywords: *context poisoning, AI coding agents, memory architecture, gated bridge, pull-request model, cross-pipeline context transfer, code-generation safety*

1 Introduction

The AI coding-agent landscape in 2026 presents a paradox: the tools that remember the most produce the best conversational experiences, while the tools that remember the least produce the most reliable code. General-purpose assistants such as ChatGPT and Claude maintain rich cross-session memory, extracting user facts, deduplicating them, and injecting them into every response. Most coding agents (including Cursor, Replit Agent, Bolt, and Lovable) maintain limited or no automatic cross-session memory, starting every session from a blank slate. This is not a coincidence; it reflects an architectural tension between conversational continuity and code-generation safety.

This paper identifies and formalizes the failure mode at the heart of that tension, context poisoning, and proposes an architectural solution, the gated memory bridge, that enables cross-session memory without exposing the generation pipeline to uncontrolled context. We implement the extraction and validation

layers of the architecture in a production system and report results from a controlled evaluation, including two pre-registered null results that refine where the failure mode actually lives.

1.1 The Two-Pipeline Problem

Modern AI coding agents are not monolithic. They contain at least two subsystems with different context requirements. The conversational subsystem answers questions, explains code, recalls history, and maintains rapport; it benefits from broad context such as user preferences, prior discussions, general knowledge, and project history. The generation subsystem produces code (scaffolds, files, migrations, tests, deployments); it requires narrow, precise context: the user’s specification, the project schema, the target framework, and relevant code patterns.

A single user session routinely mixes both modes. A user may debug an authentication endpoint (generation-relevant), ask how token refresh works in Django (educational), discuss dinner plans (irrelevant), request a generated image (irrelevant), and then say “build the fullstack app with the same layout as above” (a generation trigger that references conversational history). Any memory system that captures such a session as a flat context block and forwards it to the generation pipeline risks the generator attending to dinner plans while building a restaurant application.

1.2 Contributions

This paper makes five contributions:

1. We formalize context poisoning as a failure mode distinct from hallucination, context-window overflow, and prompt injection (Section 2).
2. We provide a comparative analysis of how five commercial systems (Cursor, Replit Agent, Claude Code, Bolt, and Lovable) handle or avoid the problem, grounded in primary sources (Section 3).
3. We propose the gated memory bridge: a typed, content-constrained, provenance-anchored, confidence-tiered interface between conversational and generation contexts (Section 4).
4. We implement the extraction and validation layers in a production system and evaluate them empirically, including a quantitative measurement of the proposer–reviewer conflict in LLM self-assessment (Sections 5–7).
5. We report two pre-registered null results on reproducing context poisoning at small scale, and reframe the gate’s measurable near-term value as structural containment and context economy rather than single-session poisoning prevention (Section 7.5).

2 Problem Definition

This section defines context poisoning, distinguishes it from adjacent failure modes, and identifies the two structural properties of coding-agent workloads that make it difficult to avoid: mixed-intent conversations and blast-radius asymmetry.

2.1 Context Poisoning

We define **context poisoning** as the corruption of a code-generation pipeline’s output caused by the injection of irrelevant, stale, or misclassified context from a conversational memory system. The divergence from user intent may manifest as:

- *irrelevant code*: for example, a `SunsetGallery` model in a restaurant app because the user generated a sunset image in the same session;

- *stale code*: for example, React 17 patterns because an outdated memory records “uses React 17” despite an upgrade to React 19;
- *contradictory code*: for example, both JWT and session-based authentication because different memories recorded different decisions;
- *scope creep*: unrequested features generated because casual conversation topics were misclassified as requirements.

2.2 Relationship to Known Failure Modes

Context poisoning is distinct from hallucination. Hallucination occurs when a model generates content not grounded in its input; context poisoning occurs when the model faithfully follows its input, but the input contains irrelevant signals. The output is grounded but misguided. It is also distinct from prompt injection, in which adversarial content manipulates model behavior: poisoning involves legitimate, user-generated content that is correctly captured but incorrectly routed.

Model capability alone does not guarantee robustness to irrelevant context, and greater capability is not automatically protective: a more capable model attends more faithfully to all provided context, including irrelevant context. Controlled studies show that irrelevant information measurably degrades model reasoning even under explicit instructions to ignore it [18], and that long contexts induce systematic positional degradation [17]. These findings motivate an architectural response: reducing uncontrolled context before it reaches the generator, rather than relying solely on the model to identify and ignore irrelevant material.

2.3 Mixed-Intent Conversations

In general-purpose chatbots, nearly every message is relevant to the user profile, so memory extraction reduces to capturing facts, deduplicating them, and injecting them everywhere. In coding agents, a single session routinely mixes project-specific work, general technical questions, creative requests, casual conversation, and meta-questions about the tool. The signal-to-noise ratio for generation-relevant memory is therefore fundamentally lower than for conversational memory.

Relevance is also not binary. “The user prefers a dark theme” is relevant to both chat (personalization) and generation (styling defaults). “The user is interested in quantum physics” is relevant to chat but toxic to generation (a naive bridge might yield a `QuantumModel`). “The user designed a blue restaurant header in an artifact” is largely irrelevant to chat memory but highly relevant to generation if the user later requests a fullstack restaurant application. Classification at extraction time cannot fully determine downstream relevance, because relevance depends on a future generation request that has not yet been made.

2.4 Blast-Radius Asymmetry

In a chatbot, an incorrect memory produces a slightly wrong response that the user corrects in the next turn. In a code-generation pipeline, incorrect context can produce code that compiles, passes surface-level checks, and ships before the defect is discovered. The blast radius of memory errors in generation is orders of magnitude larger than in conversation. This asymmetry helps explain why vendors that have successfully deployed conversational memory have been far more conservative about extending automatic memory into code generation.

3 Related Work: Memory in Commercial AI Coding Agents

No commercial coding agent currently ships automatic session-narrative memory that feeds a code-generation pipeline. This section examines how five systems arrived at that position; their design choices, and one production withdrawal, supply the empirical motivation for a gated approach.

3.1 Cursor: Automatic Memory Shipped, Then Withdrawn

Cursor is, to our knowledge, the only major AI coding agent to have shipped and subsequently withdrawn automatic cross-session memory. In May 2025 (v0.51), Cursor introduced a “Memories” feature storing user-provided and conversation-extracted facts at the project level [2]. By late 2025 (v2.1.x), the feature was removed; users were advised to export existing memories into version-controlled Rules files [3].

Three failure modes appear to have contributed to the withdrawal, based on community reports. First, privacy: the feature required disabling Cursor’s privacy mode, transmitting code to Cursor’s servers for memory extraction, which was unacceptable under enterprise data-governance policies [2]. Second, staleness: cached facts persisted after project tooling changed, causing the agent to follow outdated patterns. Third, unbounded growth: accumulated memories consumed thousands of tokens of context on every request regardless of relevance. Users reported losing extensive collections of manually curated memories when the feature was removed [3, 4]. As of July 2026, Cursor’s changelog indicates memory files can be deleted in the UI, but automatic cross-session memory has not been reintroduced [5].

The community response was to build workarounds: the “Cursor Memory Bank” project implements a documentation-driven framework of markdown files [12], and Hindsight (Vectorize) writes recalled memories into Rules files at session start [13]. Both are manual or semi-automatic, require user maintenance, and operate at the level of project conventions rather than session history.

3.2 Replit Agent: Intentional Statelessness

Replit Agent takes a deliberately stateless approach: each session is scoped by design, and the `replit.md` (or `AGENTS.md`) file provides static project instructions but stores no conversation history, session summaries, or extracted facts. Replit Agent 4 (March 2026) introduced parallel task execution with automatic merge-conflict resolution, but context remains session-scoped: parallel subagents receive isolated context, and no conversational context persists across parent sessions [6, 7].

3.3 Claude Code: Layered Persistence with Auto Memory

Anthropic’s Claude Code combines user-authored and machine-authored persistence. The primary layer is `CLAUDE.md`, a repository-committed markdown file of project instructions. In February 2026 (v2.1.59), Anthropic shipped auto memory: Claude Code writes its own operational learnings (build commands, debugging insights, conventions, workflow habits) to a local memory directory, loading the first ≈ 200 lines of `MEMORY.md` at startup and pulling topic files on demand [1, 10].

For context-poisoning analysis, the salient distinction is that Claude Code’s auto memory stores operational learnings (how to do things), not session narratives (what was discussed, decided, and left pending). Operational learnings are lower-risk because they rarely encode project decisions that can go stale; session narratives carry higher poisoning risk. Auto memory is also machine-local rather than cloud-synced [1].

3.4 Bolt and Lovable: Project-Level Context, Not Session Memory

Lovable shipped Cross-Project Referencing in February 2026, allowing read-only reuse of code, assets, and chat history from other projects in the same workspace via explicit @-mentions [8]. Lovable also maintains Workspace Knowledge and Project Knowledge, persistent instruction layers injected into LLM calls [9]. These are meaningful persistence mechanisms, but they are project-level context, not automatic session-level memory. Bolt’s public documentation, as of July 2026, does not describe a cross-session memory architecture.

3.5 Comparative Summary

Table 1 summarizes the five systems. The common pattern is avoidance: no commercial coding agent currently ships automatic session-narrative memory that feeds a code-generation pipeline. Cursor attempted the nearest equivalent and withdrew it; the remaining systems restrict persistence to user-authored instructions, operational patterns, or explicit references.

System	Memory Type	Automatic?	Poisoning Exposure	Strategy
Cursor	Cached facts (removed)	Was automatic; now manual Rules	Experienced in production	Withdrew feature
Replit	None (static replit.md)	No	Eliminated by design	Intentional statelessness
Claude Code	CLAUDE.md + auto memory	Partial (operational patterns)	Low (patterns, not decisions)	User-authored + machine-local auto
Bolt	None documented	No	N/A	No cross-session persistence
Lovable	Workspace / project knowledge	Cross-project referencing only	Low (read-only reuse)	Explicit @-references

Table 1: Memory and context-poisoning strategies in five commercial AI coding agents (as of July 2026).

3.6 Position Relative to the Research Literature

Research on memory-augmented agents has concentrated on what to remember and how to retrieve it for conversation. MemGPT introduces operating-system-style paging between a bounded context window and external storage [14]; generative agents combine a memory stream with retrieval and reflection to sustain believable long-horizon behavior [15]; and Mem0 describes production-oriented extraction and consolidation of long-term memories [16]. Our contribution is orthogonal to this line of work: rather than improving what a memory substrate stores or retrieves, the gated bridge governs which remembered content is authorized to cross into a code-generation pipeline. It can, in principle, sit atop any of these substrates.

Two further strands ground our claims. First, context-degradation studies provide the laboratory analogue of context poisoning: language models measurably lose accuracy when relevant information is embedded in long contexts [17], and are demonstrably distracted by irrelevant context even under explicit instructions to ignore it [18], supporting our argument (Section 2.2) that an architectural response is warranted rather than reliance on model improvement alone. Second, work on LLM self-evaluation documents systematic biases when models judge outputs, including their own [19], and imperfect calibration of self-knowledge [20]; our tier-enforcement finding (Section 7.2) is a production instance of the same phenomenon. Finally, the content-constraint layer (Section 4.3) responds to the threat model established by indirect prompt injection, in which data entering a model’s context carries adversarial instructions [21].

4 The Gated Memory Bridge

This section presents the proposed architecture: its design principle, an overview of the staging layer, and the four mechanisms that implement the gate (typed schemas with content constraints, confidence-tiered review, provenance anchoring, and lifecycle management with scoped supersession).

4.1 Design Principle: The Pull-Request Analogy

In software engineering, the pull request (PR) is the mechanism by which changes from a feature branch enter the main branch: changes are proposed, reviewed (automatically and/or by humans), and merged only after validation. Direct pushes to main are restricted precisely because they bypass quality controls.

We apply the same principle to memory architecture. Conversational memory is the feature branch; the generation pipeline’s context is main; the gated memory bridge (the “Memory PR”) is the controlled merge interface. No information crosses from conversation to generation without passing the gate.

4.2 Architecture Overview

The architecture introduces a staging layer between an existing conversational memory system and the generation pipeline. In the host system (Section 5), conversational memory comprises five layers: session metadata, user facts, live project knowledge, session summaries, and the current message window. The bridge sits between the conversational layers and generation-side project knowledge. Figure 1 illustrates the complete architecture: a session summarizer extracts typed proposals, a validation layer applies schema, content, and provenance checks, tiered review gates determine approval, and only approved proposals are formatted into generation context.

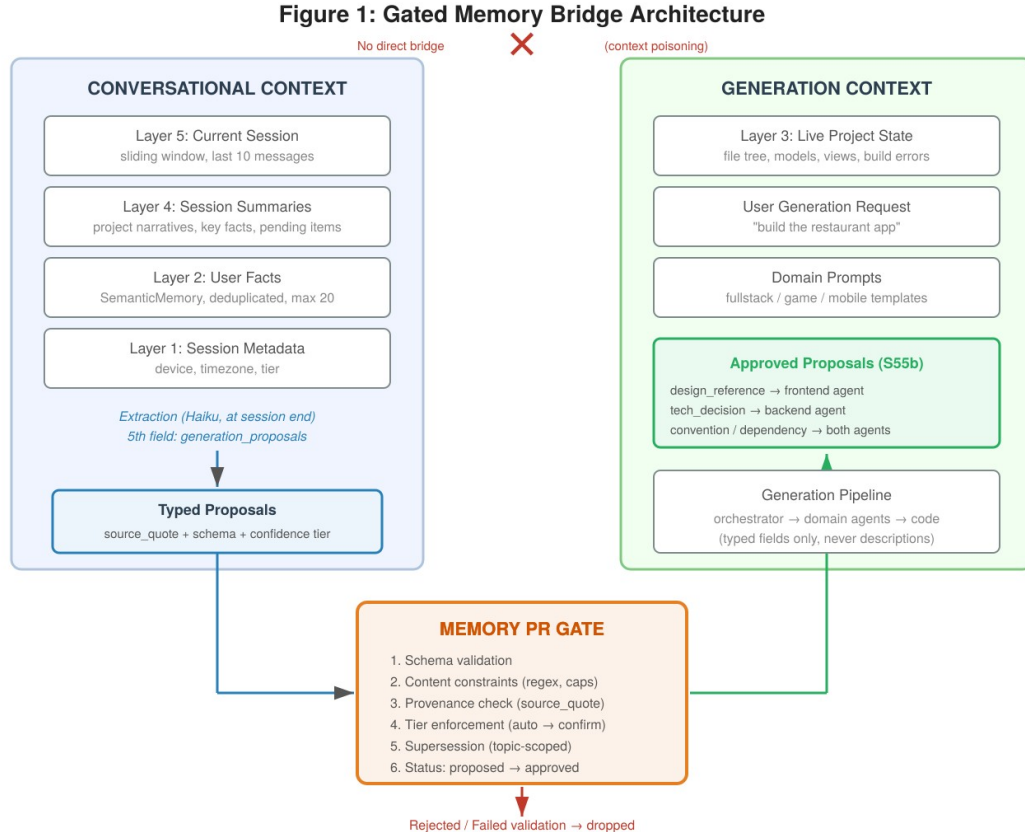


Figure 1: The gated memory bridge. Typed proposals extracted from conversation pass schema validation, content constraints, provenance verification, and tiered review before entering generation context.

4.3 Typed Schemas with Content Constraints

The gate accepts only proposals conforming to predefined typed schemas. The type system is the first filter: no generation-relevant type exists that can represent “the user is interested in quantum physics,” so such content cannot be proposed at all. We define four schema types (Table 2).

Schema Type	Required Fields	Example
design_reference	components[], description; optional color_scheme{}, layout_type	Header, MenuGrid, ReservationForm; primary color: blue
tech_decision	decision, rationale, topic (enum)	JWT authentication (topic: auth); rationale: stateless clients
project_convention	pattern, scope (enum)	/api/v1/ URL prefix; scope: backend
dependency_preference	package, reason; optional version	Stripe for payments; reason: developer experience

Table 2: Typed proposal schemas.

Typing constrains shape but not content: a string field can carry an injection payload or free-text noise, the vector exploited by indirect prompt-injection attacks [21]. Each schema therefore also enforces *content constraints*: component and package names must match identifier patterns (e.g., $^{\wedge}[A-Za-z][A-Za-z0-9_ -]\{0,63\}\$$), color values are restricted to safe character sets, string lengths are capped, and list sizes are bounded. Together, structural validation and content constraints bound both the structural shape and the lexical surface of content reaching the generation pipeline; they reduce the carrier surface for arbitrary free-text instructions, but are not claimed as a complete defense against semantic prompt injection. The `tech_decision` schema additionally carries a `topic` enumeration (auth, database, caching, api_style, deployment, testing, other) that enables scoped supersession (Section 4.6).

4.4 Confidence Tiers and Review Gates

Not all proposals warrant equal scrutiny. We define three confidence tiers (Table 3), assigned by the extractor and then deterministically enforced.

Tier	Review Process	Eligible Types
Auto-approve	Schema + content validation + provenance check; merged immediately	project_convention, dependency_preference only
Propose-and-confirm	Chatbot asks the user to confirm in a subsequent session	tech_decision, design_reference (minimum tier)
Manual review	Staged in a settings interface for explicit user review	Any type (ambiguous extractions)

Table 3: Confidence tiers and corresponding review gates.

A key design decision, validated empirically in Section 7.2, is that `tech_decision` and `design_reference` proposals are restricted to a minimum tier of `confirm` regardless of the extractor’s self-assessed confidence. This addresses the *proposer-reviewer problem*: the model that extracts a proposal also assigns its confidence, a structural conflict of interest. Our evaluation shows the extractor assigned the prohibited auto tier in 80% of restricted-type instances (Section 7.2), making deterministic enforcement essential rather than advisory.

4.5 Provenance Anchoring

Every proposal carries a `source_quote` field: the verbatim user statement from which it derives. At save time, the system verifies that the quote appears as a substring of the session’s user messages after normalization (lowercasing, punctuation stripping, whitespace collapsing). The check verifies textual presence, not semantic entailment: it rules out proposals fabricated from whole cloth, though a proposal may still misread a genuine statement. Within that scope, it converts the auto-approve tier from trust-the-model to trust-but-verify. Proposals failing provenance verification are dropped and logged.

4.6 Proposal Lifecycle and Supersession

A proposal progresses through five states: **proposed** (extracted, not yet reviewed), **approved** (passed review; available to generation), **rejected** (dismissed by the user or failed validation), **superseded** (replaced by a newer decision on the same topic), and **contradicted** (in conflict with live project state).

Supersession is governed by three rules derived during design and validated in Section 7.4. First, supersession is topic-scoped: a new caching decision supersedes only prior approved caching decisions, never unrelated authentication or database decisions. Second, supersession fires at approval time, not extraction time, so an unconfirmed proposal cannot invalidate an approved one. Third, proposals originating from the same source session are excluded from supersession candidacy: decisions made within one conversation are refinements of each other, not reversals.

4.7 A Worked Example

To make the mechanism concrete, we trace one proposal from evaluation Session A through its complete lifecycle. Steps 2 and 4 are reproduced verbatim from evaluation logs.

Step 1 – User statement. During the session, the user writes: “Use JWT authentication for this project.”

Step 2 – Extraction. At session end, the summarizer emits the proposal as structured JSON:

```
{
  "type": "tech_decision",
  "confidence": "auto",
  "source_quote": "Use JWT authentication for this project",
  "data": {
    "decision": "JWT-based authentication with token refresh",
    "rationale": "Stateless authentication for restaurant app",
    "topic": "auth"
  },
  "description": "JWT authentication chosen for the project"
}
```

Step 3 – Schema validation. The required fields (decision, rationale, topic) are present; string lengths fall within caps; the topic value is a member of the enumeration. The proposal passes structural and content validation.

Step 4 – Tier enforcement. The extractor self-assigned the auto tier, violating the prompt rule that tech_decision proposals are confirm-minimum. The deterministic enforcement layer upgrades the tier and logs the intervention. This exact upgrade occurred in the evaluation run from which this example is drawn; it is an instance of the violation rate quantified in Section 7.2.

Step 5 – Provenance verification. The source quote is normalized (lowercased, punctuation stripped, whitespace collapsed) and matched as a substring against the session’s stored user messages. The match succeeds, ruling out fabrication, though not misinterpretation (Section 4.5).

Step 6 – Review and approval. The proposal persists in state proposed at tier confirm. In the user’s next session, the assistant asks whether JWT should be used for this project; on confirmation, the proposal transitions to approved, and the supersession check runs against previously approved auth-topic decisions from other sessions.

Step 7 – Injection. When the user next triggers generation for this project, the pipeline receives only typed fields, formatted as:

APPROVED PROJECT DECISIONS:

- [auth] JWT-based authentication with token refresh
(rationale: stateless authentication)

The free-text description and the source quote are never injected; they exist for human review and audit only.

5 Implementation

The architecture was implemented in a production system. This section describes the host system, the two new modules, and the integration points, with emphasis on a deliberate property: the bridge is fully ejectable without affecting existing functionality.

5.1 Host System

The bridge was implemented in Shikshya v2, an AI agent IDE that generates fullstack Django REST + React applications from natural language, running on Google Kubernetes Engine with a Django/DRF/Celery backend. The host system already maintains a five-layer conversational memory architecture (session metadata; deduplicated user facts; live project knowledge; automatically generated per-project session summaries; and a sliding window over the current session) and a fully separate generation pipeline (orchestrator, planner, parallel domain agents, integration agent, repair pipeline) that receives no conversational memory. This pre-existing separation supplies the safety baseline; the bridge adds the controlled crossing.

5.2 New Components

Two self-contained modules were added. The **GenerationProposal model** stores typed, reviewable proposals with fields for schema type, schema-validated proposal data (JSON), a human-readable description (never injected into generation), the provenance source quote, lifecycle status, and confidence tier, with composite indexes on user/project/status and user/project/type. The **proposal schema validator** enforces three defense layers (structural validation, content constraints, and provenance verification) and contains the topic-scoped supersession logic.

5.3 Integration Points

Three existing modules received small, clearly demarcated additions. The session summarizer’s extraction prompt gained a fifth output field (generation proposals) appended after its existing four fields; per-message truncation was raised from 300 to 500 characters so decision statements late in long messages remain visible, and the output-token ceiling was raised accordingly. A save routine validates, tier-enforces, provenance-checks, and persists proposals from both the on-demand and periodic summarization paths. Finally, deleting a chat session cascades to its derived proposals. All additions are comment-demarcated; the complete ejection procedure is to delete two files, remove three delimited blocks, and revert one database migration.

6 Evaluation Methodology

We evaluate the extraction and validation layers using a pre-registered protocol: six synthetic test sessions with ground-truth labels, three extraction runs per session, a lifecycle test for supersession, and a naive-versus-gated bridging comparison with a stated contamination hypothesis.

6.1 Test Design

Five test scenarios were constructed as six chat sessions (the contradictory-decision scenario spans two sessions) under a dedicated test user isolated from production data. Complete transcripts with ground-truth labels appear in Appendix B. Each scenario targets a distinct challenge:

- **Session A (“Restaurant App”, 14 messages, ≈60% project / 40% noise).** Project work interleaved with quantum-physics questions, an image-generation request, and a dinner query. Ground truth: 2–3 proposals (UI design reference, JWT tech decision, optional reservation-form design reference).
- **Session B (“E-Commerce”, 14 messages, ≈70% / 30%).** Ground truth: 3–4 proposals (API-prefix convention, Stripe dependency preference, product-card design reference, optional localStorage tech decision).
- **Session C (“Pure Noise”, 14 messages, ≈10% / 90%).** Casual conversation, image requests, and one ambiguous mention (“maybe I should use Firebase”). Ground truth: 0–1 proposals; the ambiguous item scores as correct if extracted at manual tier or not extracted.
- **Session D (“Technical Discussion”, 12 messages, 0% decisions).** Educational discussion of REST vs. GraphQL, PostgreSQL vs. SQLite, microservices, the CAP theorem, and Kubernetes, with zero decisions. Ground truth: 0 proposals. This is the critical false-positive resistance test.
- **Sessions E1/E2 (“Contradictory Decisions”, 6 messages each).** E1: the user decides on session-based authentication. E2: the user reverses to JWT. Split across two database sessions because supersession requires a cross-session boundary.

6.2 Extraction Protocol

Each session was processed through the production summarizer three times, yielding 18 extraction runs. Raw JSON output from every run was saved to disk immediately and scored from those files rather than from database rows, avoiding confounds from the periodic summarization task. Ground truth for Session A was amended from 2 to 2–3 proposals after a pilot run (recorded as Iteration 0) revealed an unlabeled valid decision; no scored runs preceded the amendment, and the pilot output was preserved as evidence of the prompt-iteration process. Condensed per-run outputs appear in Appendix C; the full iteration log appears in Appendix G.

6.3 Scoring Rubric

- **Extraction precision:** correctly matched proposals / total proposals extracted.
- **Extraction recall (required):** required ground-truth items recovered / total required items. Items pre-marked optional are tracked separately and excluded from the required denominator.
- **Type accuracy:** strict (primary type only) and lenient (primary or a pre-registered acceptable alternative, e.g., the API-prefix item accepts both `project_convention` and `tech_decision`).
- **Confidence accuracy:** measured at two layers: the raw LLM tier assignment and the tier after deterministic enforcement. The delta quantifies the gate’s contribution.

- **Provenance validation rate:** proposals whose source quote passes substring verification / total proposals.
- **Noise leakage:** proposals traceable to off-topic conversational content (the authored noise topics) / total proposals.

6.4 Supersession Protocol

Session E1 was processed through the full save pipeline; its session-based authentication proposal was then manually approved via shell, simulating the propose-and-confirm flow (whose interactive implementation is deferred). Session E2 was then processed and its proposals approved sequentially. Three properties were verified: (1) E1’s proposal transitions to superseded when E2’s JWT proposal is approved; (2) E2’s JWT proposal remains approved when E2’s complementary refresh-token proposal is subsequently approved (same-session exclusion); (3) the final state contains exactly two approved and one superseded tech_decision.

6.5 Naive vs. Gated Comparison

For each of Sessions A–D, two conditions were constructed and submitted to a stronger generation-class model with the identical instruction to list all backend models and frontend components it would create. In the naive condition, the full conversation transcript served as generation context; in the gated condition, only approved proposals did. A pre-registered hypothesis predicted 30–50% contamination under the naive condition. A multi-session variant concatenated Sessions A, B, E1, and E2 (~830 tokens) and requested a plan for the restaurant application only, probing for cross-project contamination and stale-decision leakage. Contamination was scored as artifacts traceable to context content belonging to another topic or project; model elaboration consistent with the target application (e.g., an Order model in a restaurant app) is not contamination.

6.6 Cost

Extraction runs used a small, inexpensive model; the comparison runs used a larger model. Total evaluation cost across 33 API calls, including pilots and prompt iteration, was approximately US \$0.17 (itemized in Appendix G).

7 Results

We report extraction performance, the confidence-tier enforcement finding, type stability, supersession verification, and the naive-versus-gated comparison, including two pre-registered null results and the qualitative failure mode they surfaced. Full comparison tables appear in Appendix F.

7.1 Extraction Performance

Across 18 runs, the system extracted 31 proposals; all 31 matched ground-truth items, and zero were traceable to off-topic content (Table 4). Sessions C and D produced zero proposals across all nine of their runs: the extractor correctly distinguished an educational comparison of REST vs. GraphQL from a decision to adopt one.

Session	Run 1	Run 2	Run 3	Expected	Noise
A (Restaurant)	2	2	3	2–3	0 / 3 runs
B (E-Commerce)	4	4	4	3–4	0 / 3 runs

Session	Run 1	Run 2	Run 3	Expected	Noise
C (Pure Noise)	0	0	0	0–1	0 / 3 runs
D (Technical)	0	0	0	0	0 / 3 runs
E1 (Auth v1)	2	2	2	1–2	0 / 3 runs
E2 (Auth v2)	2	2	2	1–2	0 / 3 runs

Table 4: Extraction counts per session and run.

Aggregate metrics: extraction precision 100% (31/31); recall on required items 100% (7/7); provenance validation 100% (31/31). Because the three-run design exists to expose variance, we also report per-run figures: precision, required-item recall, and provenance validation were each 100% in every one of the 18 individual runs; observed variance was confined to optional-item recall (Table 5; item A3 in 1 of 3 runs) and to type assignment (Section 7.3). We note that after the pre-scoring ground-truth amendment gave every plausibly valid item a labeled home, precision can fail only through noise extraction; the defensible statement of this result is zero noise leakage across 31 proposals in 18 runs, on synthetic transcripts. Per-item recall (Table 5) exposes the honest variance: the optional ReservationForm decision was recovered in only 1 of 3 runs, and production systems receive a single extraction pass per session.

Item	Found	Description	Type
A1	3/3	Blue header + menu grid	design_reference
A2	3/3	JWT authentication	tech_decision
A3	1/3	ReservationForm (optional)	design_reference
B1	3/3	/api/v1/ prefix	project_convention
B2	3/3	Stripe payments	dependency_preference
B3	3/3	ProductCard component	design_reference
B4	3/3	localStorage cart (optional)	tech_decision
E1a	3/3	Session-based auth	tech_decision
E1b	3/3	Login page design (optional)	design_reference
E2a	3/3	JWT switch	tech_decision
E2b	3/3	Refresh tokens (optional)	tech_decision

Table 5: Per-item recall across three extraction runs per session.

7.2 Confidence-Tier Enforcement: The Proposer–Reviewer Conflict

The extraction prompt states explicitly that the auto tier is permitted only for project_convention and dependency_preference, never for tech_decision or design_reference. Despite this, the extractor assigned the prohibited auto tier in 20 of the 25 restricted-type instances (80%); across all 31 proposals, 20 raw assignments were incorrect (64.5%). Deterministic enforcement (a single function upgrading restricted types to confirm) raised tier accuracy from 35.5% to 93.5% (Table 6).

Metric	Value
Confidence accuracy (raw LLM assignment)	35.5%
Confidence accuracy (post-enforcement)	93.5%
Gate delta	+58.1 pp

Table 6: Confidence-tier accuracy before and after deterministic enforcement.

The interpretation is direct: the model is effective at understanding what the user decided but unreliable at assessing how much authority its own interpretation deserves. The proposing model should not be the sole authority over the privilege its proposals receive; this is consistent with documented systematic biases in LLM self-evaluation [19, 20]. The residual 6.5% error is a cascade effect: when the extractor assigns the wrong type, the type-conditional tier rule then produces the wrong tier; type errors propagate into tier errors. Both residual errors are the two type-flip instances of Session B, run 3. A per-schema-type breakdown (Appendix D) sharpens the picture: raw tier accuracy was 0% for tech_decision (all 15 instances self-assigned auto despite the explicit prohibition) and 50% for design_reference, against 100% for the two unrestricted types.

7.3 Type Stability

Type accuracy was 93.5% strict and 100% lenient. The instability source was the API-prefix item, classified as project_convention in two runs and tech_decision in one; both are defensible readings of the same statement. The $\approx 15\%$ flip rate on identical input matters for future work because type determines injection routing (conventions route to all domain agents; tech decisions route to the backend agent).

7.4 Supersession

All three verification properties passed: E1’s session-based authentication proposal was superseded upon approval of E2’s JWT proposal; E2’s JWT proposal survived the subsequent approval of E2’s refresh-token proposal; and the final state contained exactly two approved and one superseded tech_decision. The same-session exclusion proved necessary, not merely prudent: without it, E2’s two complementary same-topic decisions would have superseded each other. We state this as a design finding: topic-scoped supersession requires session-boundary exclusion to avoid invalidating complementary same-session decisions. The complete protocol trace appears in Appendix E.

7.5 Naive vs. Gated Bridging

7.5.1 Single-Session Comparison: A Pre-Registered Null

Against the pre-registered hypothesis of 30–50% contamination, the naive condition produced no detectable cross-topic contamination in any of Sessions A–D: no artifacts traceable to quantum physics, image requests, or casual conversation appeared in any generated plan. We report this null directly: modern frontier models resist single-session, mixed-intent contamination at short context lengths (≤ 14 messages). In this regime, the gate’s value is not poisoning prevention.

7.5.2 Multi-Session Comparison: A Second Null, and a Structural Guarantee

The multi-session naive condition (≈ 830 tokens of concatenated restaurant, e-commerce, and contradictory-authentication history) likewise produced no cross-project contamination and no stale-authentication leakage when asked to plan the restaurant application. This second null confirms that poisoning is not readily reproducible with short, well-structured synthetic histories; the production failures documented for Cursor involved a qualitatively different regime: unbounded context accumulated across many sessions. For the gated condition, cross-project contamination is zero by construction: only the target project’s approved proposals exist in context, so foreign-project content cannot leak regardless of history size. This containment is architectural rather than probabilistic.

7.5.3 Token Economy

The measurable near-term advantage is context size (Table 7). Naive context grows linearly with conversation history; gated context is bounded by the number of approved proposals. At production scale (dozens of sessions, thousands of messages), the naive approach consumes tens of thousands of tokens per generation request, while the gated context grows only with the active approved-decision set, which supersession and per-project decision saturation keep small in practice. This structural property makes conversation-informed generation tractable where naive dumping is unbounded.

Condition	Context Size	Ratio
Naive (multi-session transcript)	≈832 tokens	1.0×
Gated (approved proposals)	≈97 tokens	8.6× reduction

Table 7: Generation-context size, multi-session comparison. The ratio grows with history length.

7.5.4 A Qualitative Finding: Structural Hallucination

Session D exposed a failure mode the quantitative metric missed. Given the full technical discussion as naive context, the model produced a 21-artifact plan for a “tech concepts” application no one requested (TechConcept, ConceptExplorer, RelatedConceptsList, and so on): in the absence of a clear project signal, the model constructed a plausible application from ambient topical content. The gated condition, holding no approved proposals for that session, produced only a minimal four-item skeleton. We term this structural hallucination and note a complementary observation: sparser gated context also invites elaboration (the gated restaurant plan contained more invented supporting components than the naive one). The gate constrains what enters the context; it does not constrain elaboration beyond it. Constraining elaboration is a separate mechanism and future work.

8 Discussion

We consider what the results imply for the industry’s avoidance of automatic memory, for the original motivation of the gate, and for memory design in agentic systems more broadly.

8.1 Why the Industry Has Avoided This

The absence of automatic session-narrative memory in coding agents is not a capability gap: the same vendors ship conversational memory in their general-purpose assistants. The pattern across Section 3 suggests the constraint is the blast-radius asymmetry of Section 2.4. Cursor’s withdrawal reflects direct production experience; Replit’s statelessness is an explicit design choice favoring focused sessions; Claude Code confines automatic persistence to low-risk operational patterns rather than project decisions. No vendor has shipped automatic session-narrative memory that feeds a code-generation pipeline. The gated bridge is a proposal for how to do so without accepting the failure modes that motivated avoidance: privacy exposure is avoided because summaries derive from conversations already stored in the host system; staleness is bounded by topic-scoped supersession and lifecycle states; unbounded growth is bounded by the proposal cap and the constant-size injection format.

8.2 What the Results Change About the Motivation

We designed the gate to prevent poisoning and could not reproduce poisoning at the scale we could test. The honest synthesis is that the gate’s empirically demonstrated value at small scale is threefold: deterministic control over the authority of extracted context (the +58.1 pp tier-enforcement delta), verifiable provenance for every crossing item, and a generation context whose size is independent of

history length. Poisoning prevention remains the structural argument for the regime we could not test: because containment is by construction rather than by model discretion, the guarantee does not weaken as history grows, precisely the regime in which naive approaches degraded in production.

8.3 Memory and Trust in Agentic Systems

Persistent memory is frequently framed as a prerequisite for increasingly autonomous agents; recent industry benchmarking likewise treats memory as a first-class architectural component of production agent systems [11]. Our results suggest a refinement: what agents require is not memory in general but differentiated memory. Conversation calls for broad, forgiving memory; action calls for narrow, audited memory. A stateless agent is a tool; an agent that remembers is a partner; an agent that remembers indiscriminately and acts on it is a liability. The gated bridge is one instantiation of the differentiated approach.

9 Limitations

Five limitations bound the claims of this paper; the first two are the most consequential for interpreting the headline numbers.

9.1 Evaluation Scope

The evaluation uses synthetic test sessions authored by the same researcher who designed the extraction prompt (five scenarios, six sessions, 18 runs). It is a case study, not a benchmark. The transcripts were designed to cover specific edge cases and may not represent the distribution of real user behavior; in particular, decision statements were written as clean, quotable sentences, which flatters provenance verification. Real users phrase decisions elliptically (“yeah, do the JWT thing”), across multiple messages, and with typos; substring-based provenance matching will fail more often in production than the 100% observed here.

9.2 Context Poisoning at Scale

Context poisoning was not reproducible under single-session (≤ 14 messages) or short multi-session (≈ 830 -token) synthetic conditions. Cursor’s documented production withdrawal is external evidence that the failure mode exists at scale (tens of thousands of tokens of interleaved, stale, contradictory context accumulated over many sessions), a regime our evaluation could not access. We deliberately did not enlarge the synthetic histories until poisoning appeared, as iterating an experiment toward a hypothesized result would undermine the value of the pre-registered design. Production-scale evaluation with real accumulated histories is future work.

9.3 Extraction Variance

Per-item recall varies across runs on identical input (the optional ReservationForm item: 1 of 3 runs). Production systems receive one extraction pass per session, so single-shot recall for weakly stated decisions will be lower than the aggregate figures suggest. This asymmetry is by design (a missed decision costs the user a restatement, whereas a false positive risks poisoning), but the variance should be understood when interpreting the headline recall number.

9.4 Content-Constraint Trade-offs

The validation layer rejected one valid proposal during the supersession test (a design reference whose color value violated the restricted character pattern). This is a deliberate precision/recall trade: the

constraint layer prioritizes preventing injection and free-text leakage over preserving every valid proposal. We therefore distinguish extraction-level recall (what the model correctly identified) from pipeline-level recall (what survived validation), and report that the two differ.

9.5 Unimplemented Layers

This paper evaluates the extraction and validation layers. The injection layer (routing approved proposals into generation-pipeline domain agents), the interactive propose-and-confirm flow, and the user-facing review interface are implemented targets of the next development phase and were simulated where needed (Section 6.4). End-to-end effects on generated-application quality are therefore not yet measured.

10 Conclusion and Future Work

Context poisoning is an architectural risk in AI coding agents, evidenced by a production withdrawal at Cursor and industry-wide avoidance of automatic session-narrative memory for code generation. The gated memory bridge, modeled on the pull-request paradigm, provides a controlled interface between conversational and generation contexts. Our evaluation of its extraction and validation layers supports four conclusions:

1. **Precise, provenance-verified extraction is achievable.** 100% precision, 100% required-item recall, 100% provenance validation, and zero noise leakage across 18 runs on synthetic data: no off-topic conversational content reached the proposal stage.
2. **Proposer–reviewer separation is essential.** The extracting model assigned the prohibited auto tier in 80% of restricted-type instances, including every instance of the highest-risk type; deterministic enforcement contributed +58.1 percentage points of overall tier accuracy. The model that proposes context should not be the sole authority over how that context is treated.
3. **Token economy scales structurally.** The observed $8.6\times$ context reduction widens with history length, because gated context grows with the approved-decision set rather than with transcript length.
4. **Small-scale poisoning was not reproducible.** Two pre-registered attempts to reproduce context poisoning at small scale returned nulls, consistent with the hypothesis that the risk emerges with accumulated history. In that regime the gate’s protection is by construction: only approved, typed, provenance-verified proposals enter generation context, regardless of history length.

The broader implication is that memory in AI agents is not one problem but two, with conflicting requirements: conversational memory optimizes for breadth and recall, generation memory for precision and safety. The gated bridge is the interface that lets each operate fully without compromising the other. Future work comprises the injection layer, the interactive confirmation flow, constraint of elaboration beyond approved context, and evaluation at production scale on real accumulated histories.

Reproducibility Statement

The extraction prompt (Appendix A), full test-session transcripts with ground-truth labels (Appendix B), condensed per-run extraction outputs (Appendix C), scoring tables (Appendix D), the supersession trace (Appendix E), comparison data (Appendix F), and the prompt-iteration log with itemized costs (Appendix G) are included with this paper. The evaluation scripts and complete raw JSON outputs are available from the author on request.

References

- [1] Anthropic, “How Claude remembers your project,” Claude Code Documentation. <https://code.claude.com/docs/en/memory> (accessed July 2026).
- [2] Cursor Community Forum, “0.51: ‘Memories’ feature,” May 2025. <https://forum.cursor.com/t/0-51-memories-feature/98509> (accessed July 2026).
- [3] Cursor Community Forum, “Custom modes and memories gone in 2.1,” November 2025. <https://forum.cursor.com/t/custom-modes-and-memories-gone-in-2-1/143744> (accessed July 2026).
- [4] Cursor Community Forum, “Are my memories gone?,” November 2025. <https://forum.cursor.com/t/are-my-memories-gone/144057> (accessed July 2026).
- [5] Cursor, “Changelog,” <https://cursor.com/changelog> (accessed July 2026).
- [6] Replit, “Introducing Replit Agent 4: Built for Creativity,” Replit Blog, March 2026. <https://blog.replit.com/introducing-agent-4-built-for-creativity>
- [7] Replit, “Live from Replit HQ: Agent 4 Launch, Part 2,” Replit Blog, March 2026. <https://blog.replit.com/live-from-hq-agent4-launch-pt2>
- [8] Lovable, “Cross-project referencing,” Lovable Documentation. <https://docs.lovable.dev/features/cross-project-referencing> (accessed July 2026). Feature announced February 18, 2026.
- [9] Lovable, “9 ways to use Workspace Knowledge to empower your entire team,” Lovable Blog, March 2026. <https://lovable.dev/blog/9-ways-to-use-workspace-knowledge-to-empower-your-entire-team>
- [10] DataCamp, “Claude-Mem Guide: Persistent Memory for Claude Code,” March 2026. <https://www.datacamp.com/tutorial/claude-mem-guide>
- [11] Mem0, “AI Agent Memory 2026: Progress Benchmark Report,” June 2026. <https://mem0.ai/blog>
- [12] vanzan01, “Cursor Memory Bank,” GitHub repository. <https://github.com/vanzan01/cursor-memory-bank> (accessed July 2026).
- [13] Vectorize, “Hindsight: Cursor Persistent Memory,” June 2026. <https://hindsight.vectorize.io>
- [14] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.
- [15] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” in Proc. ACM UIST, 2023.
- [16] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory,” arXiv:2504.19413, 2025.
- [17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” Transactions of the ACL, vol. 12, 2024.
- [18] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, “Large Language Models Can Be Easily Distracted by Irrelevant Context,” in Proc. ICML, 2023.
- [19] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in Proc. NeurIPS Datasets and Benchmarks, 2023.
- [20] S. Kadavath et al., “Language Models (Mostly) Know What They Know,” arXiv:2207.05221, 2022.
- [21] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in Proc. ACM AISec, 2023.

Appendix A: The Extraction-Prompt Addition

The following instruction block is appended, verbatim, as the fifth output field of the production session summarizer. All extraction results in Section 7 were produced with this prompt. The identifier constraint near the end was added in the Iteration 0 revision (Section 6.2) after the pilot run produced component names such as “Header (blue)” that failed content validation.

5. "generation_proposals": An array of typed proposals for the code generation pipeline. Each proposal MUST have these fields:
 - "type": one of: design_reference, tech_decision, project_convention, dependency_preference
 - "confidence": one of: auto, confirm, manual
 - "source_quote": the EXACT verbatim user statement this derives from (copy-paste from the conversation, do not paraphrase)
 - "data": the typed fields for this schema type (see below)
 - "description": a human-readable summary (1 sentence)

Schema types and their required data fields:

- design_reference: {components: [...], description: "...", color_scheme: {...}}
- tech_decision: {decision: "...", rationale: "...", topic: "..."}
 - project_convention: {pattern: "...", scope: "backend|frontend|api|naming|structure"}
- dependency_preference: {package: "...", reason: "..."}
 -

Confidence rules:

- "auto" ONLY for project_convention and dependency_preference when the user made an explicit, unambiguous statement. NEVER auto for tech_decision or design_reference.
- "confirm" for tech_decision, design_reference, or any inferred proposal.
- "manual" for ambiguous or low-confidence extractions.

ONLY extract when the user made a clear statement or action about how the project should be built. Do NOT extract from:

- General technical questions ("how does JWT work?")
- Casual conversation, image/video generation, debugging discussions
- Comparisons or explorations ("should we use X or Y?") - these are questions, not decisions

Component names must be valid identifiers (PascalCase, no spaces or punctuation, e.g., MenuGrid, not "MenuGrid (3-column)"); place colors in color_scheme and layout details in description.

When uncertain, do NOT extract. False negatives are cheap. False positives are context poisoning. Return empty array if nothing qualifies.

Appendix B: Test Session Transcripts

Five test scenarios (six database sessions) were created under a dedicated test user (shikshya_eval_user), isolated from production data. Each message records its role and project context; an empty project field denotes off-topic or general conversation. Ground-truth labels follow each transcript; ✓ marks items that should be extracted, ✗ marks items that must not be.

B.1 Session A – Restaurant App (14 messages; ≈60% project, 40% noise)

#	Role	Project	Message
1	user	tastybites	I want to build a restaurant app called TastyBites
2	assistant	tastybites	Great! I can help you build TastyBites. What features are you thinking?
3	user	tastybites	Make the header blue with a 3-column menu grid for the main page
4	assistant	tastybites	I'll design a blue header with a 3-column responsive menu grid. Should I include filtering?
5	user	tastybites	Use JWT authentication for this project
6	assistant	tastybites	Got it, I'll set up JWT-based auth with token refresh.
7	user	(none)	Hey quick question, how does quantum entanglement work with photons?
8	assistant	(none)	Quantum entanglement occurs when two particles become correlated...
9	user	(none)	Generate me a beautiful sunset image over the ocean
10	assistant	(none)	I'll generate that sunset image for you.
11	user	(none)	What should I have for dinner tonight? I'm thinking Italian or Thai
12	assistant	(none)	Both are great options! For Italian you could try...
13	user	tastybites	Back to the restaurant app, also add a reservation form component
14	assistant	tastybites	I'll add a ReservationForm component with date, time, and party size fields.

Table B1: Session A transcript.

Ground truth (amended after pilot; Appendix G):

- ✓ design_reference (confirm): blue header + 3-column menu grid + color scheme. Source: message 3.
- ✓ tech_decision (confirm): JWT authentication, topic auth. Source: message 5.
- ✓ design_reference (confirm, optional): ReservationForm component. Source: message 13.
- ✗ Quantum physics (7), sunset image (9), dinner suggestion (11): noise; must not be extracted.

B.2 Session B – E-Commerce (14 messages; ≈70% project, 30% noise)

#	Role	Project	Message
1	user	shopwave	Building an e-commerce platform called ShopWave
2	assistant	shopwave	Let's build ShopWave. What kind of products will it sell?
3	user	shopwave	All endpoints should use the /api/v1/ prefix for versioning
4	assistant	shopwave	I'll configure all API routes under /api/v1/. Good practice for versioning.
5	user	shopwave	Use Stripe for payment processing, it has the best developer experience
6	assistant	shopwave	Stripe is a great choice. I'll integrate Stripe Checkout and webhooks.
7	user	(none)	What are some good hiking trails near San Francisco?
8	assistant	(none)	There are many great trails! Lands End, Dipsea Trail in Muir Woods...
9	user	shopwave	The product card should show image, title, price, and add-to-cart button
10	assistant	shopwave	I'll design the ProductCard component with those four elements.
11	user	(none)	What's the weather like in Tokyo right now?

#	Role	Project	Message
12	assistant	(none)	I don't have real-time weather data, but Tokyo in July is typically warm and humid.
13	user	shopwave	Also make sure the cart persists across page refreshes using localStorage
14	assistant	shopwave	I'll implement cart persistence via localStorage with a useCart hook.

Table B2: Session B transcript.

Ground truth:

- ✓ project_convention (auto): /api/v1/ prefix, scope api. Source: message 3. Acceptable alternative type: tech_decision.
- ✓ dependency_preference (auto): Stripe for payments. Source: message 5.
- ✓ design_reference (confirm): ProductCard with image, title, price, add-to-cart. Source: message 9.
- ✓ tech_decision (confirm, optional): localStorage cart persistence. Source: message 13. Acceptable alternative type: project_convention.
- ✗ Hiking trails (7), Tokyo weather (11): noise; must not be extracted.

B.3 Session C – Pure Noise (14 messages; ≈10% project, 90% noise)

#	Role	Project	Message
1	user	(none)	Hey what's up, how are you doing today?
2	assistant	(none)	I'm doing well! How can I help you today?
3	user	(none)	Can you explain the theory of relativity in simple terms?
4	assistant	(none)	Sure! Einstein's theory of relativity comes in two parts...
5	user	(none)	Generate me an image of a cat wearing a top hat
6	assistant	(none)	I'll generate that image for you!
7	user	(none)	What's the meaning of life according to different philosophies?
8	assistant	(none)	Different philosophical traditions approach this question uniquely...
9	user	(none)	Maybe I should use Firebase for my next project, not sure yet though
10	assistant	(none)	Firebase is a good option for rapid prototyping with its real-time database...
11	user	(none)	Tell me a joke about programmers
12	assistant	(none)	Why do programmers prefer dark mode? Because light attracts bugs!
13	user	(none)	What's the best way to learn guitar as an adult?
14	assistant	(none)	Start with basic chords like G, C, D, and Em...

Table B3: Session C transcript.

Ground truth:

- ? dependency_preference (manual, optional): Firebase. Source: message 9. Ambiguous: “maybe” and “not sure yet” indicate no decision was made; extracted-as-manual and not-extracted both score as correct. The empty project field additionally triggers the project-name guard.
- ✗ All other messages: noise; must not be extracted.

B.4 Session D – Technical Discussion (12 messages; 0% decisions)

#	Role	Project	Message
1	user	(none)	Can you explain the differences between REST and GraphQL APIs?
2	assistant	(none)	REST and GraphQL take fundamentally different approaches. REST uses resource-based URLs...
3	user	(none)	What are the tradeoffs between PostgreSQL and SQLite for small projects?
4	assistant	(none)	PostgreSQL offers full ACID compliance, concurrent connections... SQLite is zero-config...
5	user	(none)	How do microservices communicate? I've heard about event-driven patterns
6	assistant	(none)	Microservices can communicate synchronously via REST/gRPC or asynchronously via message queues...
7	user	(none)	What's the difference between horizontal and vertical scaling?
8	assistant	(none)	Vertical scaling means upgrading a single server. Horizontal scaling means adding more servers...
9	user	(none)	Tell me about the CAP theorem and how it applies to distributed databases
10	assistant	(none)	The CAP theorem states that a distributed system can only guarantee two of three properties...
11	user	(none)	How does container orchestration work with Kubernetes?
12	assistant	(none)	Kubernetes manages containerized applications across a cluster of machines...

Table B4: Session D transcript. Every message is technical; none is a decision.

Ground truth: zero proposals. All items are educational comparisons or general knowledge. A high extraction rate here indicates poor precision; this is the critical false-positive resistance test.

B.5 Session E1 – Auth Decision v1 (6 messages)

#	Role	Project	Message
1	user	securevault	Starting a new project called SecureVault for password management
2	assistant	securevault	SecureVault sounds like a great project! Security will be critical here.
3	user	securevault	Use session-based authentication for this project, I think cookies are more secure
4	assistant	securevault	Session-based auth with httpOnly cookies is indeed a solid choice for security.
5	user	securevault	The login page should have email and password fields with remember me checkbox
6	assistant	securevault	I'll design the login form with those three elements plus proper validation.

Table B5: Session E1 transcript.

Ground truth:

- ✓ tech_decision (confirm): session-based authentication, topic auth. Source: message 3.
- ✓ design_reference (confirm, optional): login page with email, password, remember-me. Source: message 5.

B.6 Session E2 – Auth Decision v2 (6 messages)

#	Role	Project	Message
1	user	securevault	Back to SecureVault, I've been thinking about the auth approach
2	assistant	securevault	Sure! Last time we discussed session-based auth. What are you thinking?
3	user	securevault	Actually, let's switch to JWT instead of session-based auth, it's better for mobile clients
4	assistant	securevault	JWT is a good choice for mobile compatibility. I'll update the auth strategy.
5	user	securevault	Make sure we use refresh tokens with short-lived access tokens for security
6	assistant	securevault	I'll implement the refresh token pattern with 15-minute access tokens.

Table B6: Session E2 transcript.

Ground truth:

- ✓ tech_decision (confirm): JWT authentication (supersedes E1), topic auth. Source: message 3.
- ✓ tech_decision (confirm, optional): refresh-token pattern, topic auth. Source: message 5.

Supersession requirement: after E1's proposal is approved and E2's JWT proposal is subsequently approved, E1's session-based decision must transition to superseded; E2's refresh-token proposal must not supersede E2's JWT proposal (same-session exclusion).

Appendix C: Raw Extraction Outputs

Condensed per-run outputs; complete raw JSON for all 18 runs accompanies the supplementary materials. "Raw" denotes the extractor's self-assigned confidence tier before deterministic enforcement.

C.1 Session A

Field	Proposal 1	Proposal 2
type	design_reference	tech_decision
confidence (raw)	auto	auto
confidence (enforced)	confirm	confirm
source_quote	"Make the header blue with a 3-column menu grid for the main page"	"Use JWT authentication for this project"
key data	components: Header, MenuGrid; color_scheme: primary=blue	decision: JWT-based auth with token refresh; topic: auth

Table C1: Session A, Run 1 (2 proposals, 3.6 s).

Run 2 (2 proposals, 2.9 s): identical structure; the design_reference raw tier was confirm (correct) while the tech_decision raw tier was again auto (enforced to confirm). Run 3 (3 proposals, 3.3 s): additionally extracted ReservationForm as a separate design_reference; raw tiers confirm/auto/auto, all enforced to confirm. Noise rejection: the quantum-physics, sunset-image, and dinner messages were extracted in none of the three runs.

C.2 Session B

Field	Proposal 1	Proposal 2	Proposal 3	Proposal 4
type	project_convention	dependency_preference	design_reference	tech_decision
raw tier	auto	auto	auto	auto
enforced tier	auto	auto	confirm	confirm
source (msg)	3: /api/v1/ prefix	5: Stripe	9: product card	13: localStorage cart

Table C2: Session B, Run 1 (4 proposals, 4.2 s).

Run 2 (4 proposals, 4.1 s): same four proposals; the design_reference raw tier was confirm (correct in this run). Run 3 (4 proposals, 4.9 s): type instability observed; the /api/v1/ item was classified tech_decision (project_convention in runs 1–2) and the localStorage item project_convention (tech_decision in runs 1–2); both alternatives are acceptable under the pre-registered rubric (Section 6.3). Noise rejection: hiking and weather messages extracted in no run.

C.3 Sessions C and D

Both sessions produced zero proposals in every run (six runs total, zero noise). The ambiguous Firebase mention in Session C was not extracted in any run, a correct outcome given the hedging language and the empty-project-name guard. Session D’s educational content (REST vs. GraphQL, CAP theorem) was never converted into decisions. Notably, the raw outputs show that the Firebase mention and off-topic interests were captured in the summarizer’s conversational user-facts channel in every run, evidence that the dual-output design routes ambiguity to the low-risk memory layer rather than to the generation bridge.

C.4 Sessions E1 and E2

E1 (all three runs, 2 proposals each): the tech_decision (session-based auth, topic auth) carried raw tier auto in all runs; the optional design_reference (login page) carried raw tier confirm in runs 1 and 3 and auto in run 2; all enforced to confirm. E2 (all three runs, 2 proposals each): both tech_decisions (JWT switch; refresh tokens, both topic auth) carried raw tier auto in every run, all enforced to confirm.

Appendix D: Scoring Tables

D.1 Aggregate Metrics Against Pre-Registered Criteria

Metric	Value	Criterion	Status
Extraction precision	100.0% (31/31)	$\geq 80\%$	Pass
Extraction recall (required)	100.0% (7/7)	$\geq 70\%$	Pass
Provenance validation rate	100.0% (31/31)	$\geq 90\%$	Pass
Session D proposals (max/run)	0	≤ 1	Pass
Type accuracy (strict / lenient)	93.5% / 100.0%	reported	–
Confidence accuracy (raw / enforced)	35.5% / 93.5%	reported	–
Gate delta	+58.1 pp	reported	headline

Table D1: Aggregate metrics. Per-item recall appears as Table 5 in the main text.

D.2 Confidence Tier Analysis by Schema Type

Schema type	Instances	Raw auto assigned	Expected tier	Raw accuracy
design_reference	10	5 (50%)	confirm	50%
tech_decision	15	15 (100%)	confirm	0%
project_convention	3	3 (100%)	auto	100%
dependency_preference	3	3 (100%)	auto	100%

Table D2: Raw tier assignment by schema type, computed over raw type assignments per run. Totals reconcile exactly with the aggregate: 11 of 31 raw assignments correct (35.5%).

The pattern is stark: the extractor assigns correct tiers for the two low-risk types in every instance, yet assigns auto to the high-risk tech_decision type in 100% of instances (15 of 15) despite an explicit prompt prohibition. The deterministic enforcement layer is therefore not a backstop; it is the primary mechanism of tier safety.

Appendix E: Supersession Test Trace

E.1 Protocol Steps

(1) Session E1 processed through the full save pipeline; one proposal saved as tech_decision/confirm/proposed (session-based auth). The session’s optional design_reference was rejected at save time by a content constraint (E.3). (2) E1’s proposal manually approved via shell, simulating the deferred confirm flow. (3) Session E2 processed; two proposals saved (JWT switch; refresh tokens), both tech_decision/confirm/proposed. (4) E2’s JWT proposal approved; the supersession check found E1’s approved same-project, same-type, same-topic decision from a different session and marked it superseded (count: 1). (5) E2’s refresh-token proposal approved; the check excluded E2’s own JWT proposal via the same-session exclusion (count: 0).

E.2 Final State and Verification

Session	Type	Status	Description
E1	tech_decision	superseded	Session-based auth with httpOnly cookies
E2	tech_decision	approved	JWT-based authentication
E2	tech_decision	approved	Refresh tokens, 15-minute access tokens

Table E1: Final proposal states. All three verification properties of Section 6.4 passed.

E.3 Content-Constraint Rejection

E1’s design_reference was rejected at save time with the log message “color_scheme[primary]: value does not match pattern”; the extractor likely produced a multi-word color value that failed the restricted character pattern. This is the deliberate precision/recall trade-off discussed in Section 9.4 and the reason extraction-level and pipeline-level recall are reported separately.

Appendix F: Naive vs. Gated Comparison Data

F.1 Single-Session Results

Session	Naive artifacts	Gated artifacts	Contamination
A (Restaurant)	12	22	none in either condition
B (E-Commerce)	40	27	none in either condition
C (Pure Noise)	0	4 (generic scaffold)	none
D (Technical)	21 (structural hallucination)	4 (generic scaffold)	see note

Table F1: Single-session comparison. Contamination is defined as artifacts traceable to off-topic context content; model elaboration consistent with the target application does not count.

Session D note: under naive bridging the model constructed a 21-artifact “tech concepts” application (TechConcept, ConceptExplorer, RelatedConceptsList, and so on) from an educational discussion containing zero decisions. This is the structural-hallucination failure mode analyzed in Section 7.5.4, distinct from cross-topic contamination; excluding Session D, the single-session poisoning rate was zero in both conditions, the first pre-registered null.

F.2 Multi-Session Results

Condition	Context tokens	Artifacts	Bleed (cross-project / stale auth)
Naive (A+B+E1+E2)	≈832	18	0 / 0
Gated (approved only)	≈97	31	0 / 0 (by construction)

Table F2: Multi-session comparison, instruction “build the TastyBites restaurant app.” Second pre-registered null; the test is underpowered relative to production-scale histories (Section 9.2).

F.3 Token Economy

Naive context grows linearly with accumulated history; gated context is bounded by approved-proposal count. The observed ratio was $8.6\times$ at four sessions of history. The ratio widens as history accumulates, because the gated side grows with the active approved-decision set rather than with transcript length; quantifying it at production scale requires the longitudinal data identified as future work.

Appendix G: Prompt Iteration Log and Evaluation Cost

G.1 Iteration 0 (pilot run, pre-scoring)

Issue 1: component names included attributes (“Header (blue)”, “MenuGrid (3-column)”), failing the identifier pattern. Fix: the identifier constraint reproduced at the end of Appendix A was added to the prompt. Issue 2: every proposal in the pilot self-assigned the auto tier, including restricted types; the enforcement layer upgraded them correctly at save time, and this observation became the headline finding of Section 7.2. Issue 3: the pilot extracted a valid third decision (ReservationForm) absent from Session A’s ground truth; the ground truth was amended to 2–3 before any scored run, and the pilot output was preserved.

G.2 Iteration 1 (verification, pre-scoring)

One additional pilot on Session A verified the fix: components were emitted as PascalCase identifiers, the color scheme was populated (primary: blue), and the design_reference raw tier was correctly assigned confirm. Scored runs then proceeded with the prompt frozen.

G.3 Evaluation Cost

Phase	Calls	Model class	Estimated cost (USD)
Pilot runs	2	small (extraction)	\$0.013
Baseline comparison	1	small (extraction)	\$0.006
Scored extraction runs	18	small (extraction)	\$0.060
Supersession test	2	small (extraction)	\$0.012
Naive vs. gated (single)	8	large (generation)	\$0.040
Naive vs. gated (multi)	2	large (generation)	\$0.010
Total	33	–	≈\$0.17

Table G1: Itemized evaluation cost.